# LLMKernelBench: Benchmarking Large Language Models on Software Vulnerability Detection in Linux Kernel

Arastoo Zibaeirad, Rodrigo Pato Nogueira, Marco Vieira
University of North Carolina at Charlotte, Charlotte, NC, USA
Email: {azibaeir, rpatodec, marco.vieira}@charlotte.edu

***Abstract*—Large Language Models (LLMs) demonstrate strong capabilities in code-related tasks, however their effectiveness in Software Vulnerability Detection (SVD) remains poorly understood due to inadequate evaluation frameworks. Existing benchmarks suffer from training data contamination, isolated function evaluation without cross-component context, lack of strict vulnerable-patched pairing, and binary classification without hierarchy-aware Common Weakness Enumeration (CWE) assessment, which prevents a reliable measurement of security reasoning versus pattern matching on leaked data. We introduce LLMKernelBench, a rigorous benchmark comprising 417 real-world Linux kernel vulnerabilities across 74 CWE types, split into a Primary Benchmark Dataset (PBD; 314 samples, $\leq$2024) and a Leakage Free Dataset (LFD; 103 samples, 2025 post-cutoff), with context-aware evaluation at three granularity levels and hierarchy-aware metrics quantifying semantic proximity in misclassifications. We evaluate seven LLMs spanning code-specialized and general-purpose architectures. Binary vulnerability detection is near-random ($\sim 50\%$ accuracy) and strongly biased: some models label $> 75\%$ of samples as vulnerable, while others mostly label them as non-vulnerable. CWE prediction is effectively unusable, with an average Top-1 accuracy of $1.26\%$ (best: $2.92\%$) and a $10.59\%$ hierarchy proximity score, indicating almost no understanding of vulnerability families. On the leakage-free 2025 split, general-purpose models degrade minimally ($0.3$ percentage points), while code-specialized models drop sharply ($8.8$ points), suggesting that code-specialized models exhibit behavior consistent with memorization-based pattern matching rather than robust security reasoning.**

***Index Terms*—Large Language Models, Software Security, Software Vulnerability Detection**

## I. Introduction

Software vulnerabilities pose a critical threat to modern computing infrastructure. The number of identified CVEs has surged to over 29,000 in 2023, up from 20,153 in 2021 [1], underscoring the urgent need for automated Software Vulnerability Detection (SVD). Real-world vulnerabilities rarely manifest in isolated code fragments but instead involve intricate interactions across multiple functions, files, and modules. Detecting these requires not only identifying whether code is vulnerable but also classifying the specific Common Weakness Enumeration (CWE) category (which is essential for triage, prioritization, and linking to mitigation strategies). As production systems such as the Linux kernel ($\sim$30M LOC) power Android, servers, and embedded devices, and because diverse, high-impact flaws persist, effective automated detection is critical for software security at scale.

Traditional SVD approaches include Automated Program Repair (APR), fuzzing, and Static Analysis Tools (SATs), but these face fundamental limitations: patch quality issues [2], [3], coverage gaps and logic error blindness [4], [5], and high false positive rates [6]. Recent work has turned to Large Language Models (LLMs) [7]–[11], which demonstrate strong capabilities in code understanding, test generation [12], [13], and various programming tasks [14]. Multiple benchmarks have emerged to evaluate LLM security reasoning, incorporating vulnerability-specific metadata [15]–[17] and CWE-wise analysis [18], [19].

Despite progress, existing benchmarks exhibit critical limitations. First, most rely on historical data that likely overlaps with LLM training corpora, artificially inflating metrics through *training data contamination*, with only limited efforts [19] addressing this via post-cutoff validation. Second, datasets often extract *isolated functions* without surrounding context (macros, type definitions, cross-file dependencies), failing to capture how real vulnerabilities manifest through component interactions. Third, many lack *strict vulnerable-patched pairing*, instead collecting functions across different commits or versions, introducing label noise [20]–[22]. Fourth, evaluations focus on binary classification without assessing *CWE hierarchy understanding* (whether models grasp vulnerability families or merely memorize surface patterns). These gaps prevent a reliable assessment of whether LLMs possess genuine security reasoning versus pattern matching on contaminated data.

In this paper, we present LLMKernelBench, a rigorous benchmark addressing these limitations through three key design principles. First, we introduce a *temporal split*: the Primary Benchmark Dataset (PBD) covers vulnerabilities through 2024, while the Leakage Free Dataset (LFD) contains only 2025 post-cutoff CVEs, enabling direct measurement of generalization versus memorization. Second, we adopt a *context-aware* approach, pairing 417 real-world Linux kernel CVEs with strict vulnerable-patched pairing while preserving surrounding non-functional elements that influence vulnerability semantics. Third, we evaluate across three *abstraction levels* (single function, multiple functions, multiple files) to assess context utilization, which is essential for deployment but has not been explored in prior work. We evaluate seven pre-trained LLMs spanning code-specialized (CodeLlama, StarCoder2,

Qwen2.5-Coder) and general-purpose models (Llama3.1, Mistral, DeepSeek-R1, GPT-4.1-mini) across 74 CWE types, measuring both binary detection and hierarchy-aware CWE classification through novel metrics (Hierarchical Proximity Score (HPS)) that reward semantic proximity.

Our work intentionally targets the Linux kernel as a representative example of a *high-assurance, security-critical software system*. The kernel's extensive use of macros, low-level memory management, and long-evolving coding conventions reflects the complexity of infrastructure software, where vulnerabilities have severe consequences. Accordingly, LLMKernelBench is not designed to characterize LLM performance on general-purpose application code, but rather to stress-test vulnerability-detection capabilities under conditions representative of real-world, safety- and security-critical systems.

Our evaluation addresses four research questions:

- **RQ1: Binary SVD performance**: How effectively can large language models detect whether a code sample is vulnerable or non-vulnerable?
- **RQ2: Consistency across CWEs pillars**: Are model performances consistent across different CWEs pillars?
- **RQ3: Consistency across granularity levels**: Are model performances consistent across different code granularities (file-, function-, and multi-file-level)?
- **RQ4: Vulnerability type identification**: When a vulnerability is detected, how accurately can models identify its corresponding CWE category?

**Contributions.** In summary, this work offers:

- **Leakage-aware benchmark with temporal split.** We introduce LLMKernelBench, the first kernel-focused benchmark with systematic contamination control, including 417 real-world Linux kernel CVEs split into PBD (314 samples, $\leq$2024) and LFD (103 samples, 2025 post-cutoff), enabling direct measurement of generalization versus memorization.
- **Context-aware evaluation at three abstraction levels.** Unlike prior work using isolated functions, we provide strict vulnerable-patched pairing with surrounding context (macros, type definitions, globals) and evaluate at L1 (single function), L2 (multiple functions), and L3 (multiple files) to assess whether models leverage cross-component dependencies.
- **Hierarchy-aware CWE evaluation metric.** We develop the HPS metric combining Top-$k$ accuracy, Mean Reciprocal Rank (MMR), and CWE taxonomy structure to quantify semantic proximity in misclassifications, distinguishing whether models confuse similar weakness families or make random errors.
- **Comprehensive evaluation of seven LLMs.** We evaluate code-specialized (CodeLlama, StarCoder2, Qwen2.5-Coder) and general-purpose models (Llama3.1, Mistral, DeepSeek-R1, GPT-4.1-mini) across 74 CWE types. We focus on open-source models to ensure transparency, reproducibility, and cost-effectiveness.

From a reliability engineering perspective, vulnerability detection models act as *diagnostic components* within a broader software assurance pipeline. Their practical value depends not only on average accuracy, but also on predictable behavior under realistic operating conditions, robustness to unseen inputs, and stable failure modes when confidence is unwarranted. Our results show that current LLM-based approaches fail to meet these reliability requirements: they exhibit near-random decision accuracy, extreme class bias, high variance across abstraction levels, and sharp performance degradation on leakage-free data. These behaviors indicate that, in their current form, LLMs cannot be relied upon as dependable vulnerability detectors in safety- or security-critical systems. By systematically characterizing these failure modes under controlled, leakage-aware conditions, this work provides empirical evidence for understanding the reliability limits of LLM-based analysis tools and serves as a benchmark for assessing future improvements in dependable AI-assisted software assurance.

The remainder of this paper is structured as follows: **§II** reviews background and related work. **§III** presents the benchmark and evaluation protocol. **§IV** reports findings for RQ1–RQ4. **§V** discusses implications and limitations. **§VI** summarizes threats to validity. **§VII** concludes the paper. Appendix A provides further motivating examples and task definitions.

## II. Background and Related Work

*CWE Taxonomy and Hierarchical Evaluation:* The CWE system [23] organizes software weaknesses into a four-level hierarchy: Pillars, Classes, Base weaknesses, and Variants. This enables graduated assessment, where identifying a parent category demonstrates partial understanding even when the specific variant is misidentified. Our work leverages this hierarchy to measure both exact classification accuracy and semantic proximity through the HPS metric.

*Vulnerability Datasets – From File-Level to Context-Aware:* Early datasets adopted *file-level* labeling, marking entire files touched by security commits as vulnerable [24]–[27]. While enabling large-scale collection, this introduced label noise, class imbalance [20], and difficulty distinguishing security fixes from refactoring [21], [22]. The field shifted toward *function-level* granularity [28], [29], but these often failed to strictly pair vulnerable-patched versions of the *same* function or account for cross-component dependencies.

*Unlike* prior work extracting isolated functions, LLMKernelBench adopts a *context-aware* approach, pairing line-modified functions with surrounding non-functional elements (macros, type definitions, globals) that influence vulnerability semantics. By evaluating at three abstraction levels (single function, multiple functions, multiple files), we assess whether LLMs can leverage broader context, essential for deployment but unexplored in existing benchmarks.

*LLM Benchmarks – From Scale to Quality and Leakage Control:* Early benchmarks prioritized *scale*, collecting tens of thousands of samples but lacking vulnerability-specific

TABLE I: VD benchmarks at a glance

| Benchmark | Scale | CWE | Pairs (vuln vs patch) | Leakage | Granularity |
|---|---|---|---|---|---|
| [30] (2019) | 27,318 | – | ✗ | ✗ | Func |
| [15] (2022) | 130 | 75 | ✓ | ✗ | Snippet |
| [16] (2024) | 1000 | 48 | ✓ | ✗ | Program |
| [17] (2025) | 294 | 77/86 | ✓ | partial | Func. |
| [18] (2025) | 5,000 | 25 | partial | ✗ | Func.+project |
| [19] (2024) | 228 | 8 | ✓ | ✓ | Func. |
| LLMKernelBench | 417 | 74 | ✓ | ✓ | Func.+context |

metadata [28], [30]. The field evolved toward richer annotations [15]–[17] and CWE-wise analysis [18], [19], yet a critical gap remains: *training data contamination*. Most benchmarks rely on historical data that likely overlaps with model training corpora, artificially inflating metrics. Only limited efforts [19] have considered post-cutoff validation, yet none have used systematic temporal splits.

LLMKernelBench addresses these limitations by prioritizing *quality over scale* and *leakage-aware* evaluation (Table I). We provide 314+103 curated CVE-linked samples across 74 CWE types with strict vulnerable-patched pairing. Critically, we introduce a temporal split: PBD (covering CVEs through 2024) versus LFD (covering 2025 post-training-cutoff CVEs), enabling direct measurement of generalization versus memorization. Our hierarchy-aware evaluation and context-aware abstraction levels position LLMKernelBench as a *stress-test benchmark* for security reasoning, complementing broader training corpora while addressing systematic limitations in rigor and contamination control.

## III. Benchmarking Approach

LLMKernelBench is designed to assess pre-trained LLMs (see Table IV for the models evaluated) on SVD and CWE-label classification tasks, focusing on C code from the Linux kernel while being extensible to other programming languages. As illustrated in Figure 1, the framework is based on vulnerability metadata and code pairs extracted from Linux CVE commits, and includes prompts designed for SVD and CWE-label classification tasks. LLMKernelBench evaluates the models in a zero-shot setting with temperature set to 0 to ensure deterministic and reproducible outputs, using binary classification metrics (precision, recall, accuracy, F1) for SVD and ranking metrics for CWE-label classification. This approach ensures benchmark reliability through representativeness (diverse real-world vulnerabilities), adaptability (an extendable methodology), portability (applicability across various LLMs), and fairness (standardized evaluation procedures).

Section III-A details our dataset construction process. Section III-C defines metrics for measuring effectiveness in SVD and CWE-label classification tasks. Finally, Section III-D presents our standardized prompt templates.

### A. Dataset

Our method systematically extracts pairs of vulnerable code blocks and their corresponding patched versions from real-world Linux kernel commits. We focus on the Linux kernel due to its rich vulnerability history and critical security importance. We collect commit hashes, CVE identifiers, and CWE classifications from well-known vulnerability databases, including NVD [31], in line with established practices [32]. For each vulnerability-fixing commit, we retrieve the vulnerable code from its parent commit (pre-fix version) and the patched code from the commit itself (post-fix version).

Our extraction process identifies all security-related functions and non-functional elements (global variables, macros, type definitions, structs/unions, headers) that were directly modified in each commit. We include only functions with actual line-level changes, excluding unchanged code to avoid label noise. Non-functional elements are tracked separately as they often form critical vulnerability contexts that influence execution across multiple functions. This comprehensive approach assembles cohesive, vulnerable, and patched code blocks at multiple abstraction levels (single function, multiple functions, multiple files), enabling LLMs to analyze complete contextual information rather than isolated fragments.

After extraction, the dataset is divided into two. The PBD covers all vulnerabilities discovered on or before 2024, while the LFD covers the more recent 2025 vulnerabilities. This division enables us to evaluate the impact of potential data leakage on the model's performance, thereby increasing our confidence in the generalizability of our findings.

**Manual Review, Denoising, and Deduplication.** Our automated extraction process allowed us to collect a total of 400 samples for PBD and 110 samples for LFD. After that, to ensure dataset integrity and label fidelity, both datasets were manually validated by two different reviewers. For this validation, a standardized protocol that defined inclusion criteria, exclusion criteria, and required evidence documentation was first defined. Then, the original datasets were split in half, and each half was assigned to a different reviewer. The reviewers followed the previously defined documentation to clean the dataset. Doubtful cases were marked for discussion and resolved through consensus sessions between the reviewers. The following validations were performed:

- **Denoising**: Removed samples and functions/files where the changes were not security-related.
- **Completeness check**: Ensured all relevant and security-impacting changes from each commit were included.
- **CWE frequency filtering**: CWEs that appeared only once were removed to improve representativeness.
- **Comment removal**: Stripped all code comments from both vulnerable and patched code blocks to eliminate potential biases and ensure that the LLMs relied solely on code structure and semantics for SVD.
- **Deduplication**: Removed duplicate or near-duplicate code samples (e.g., identical functions, commits, or cloned files) to prevent models from seeing the same or highly similar code during testing.
- **CWE labeling**: In the LFD, 40 samples lacked CWE identifiers in the NVD database; we manually labeled these samples, strictly adhering to NVD criteria for consistent labeling.

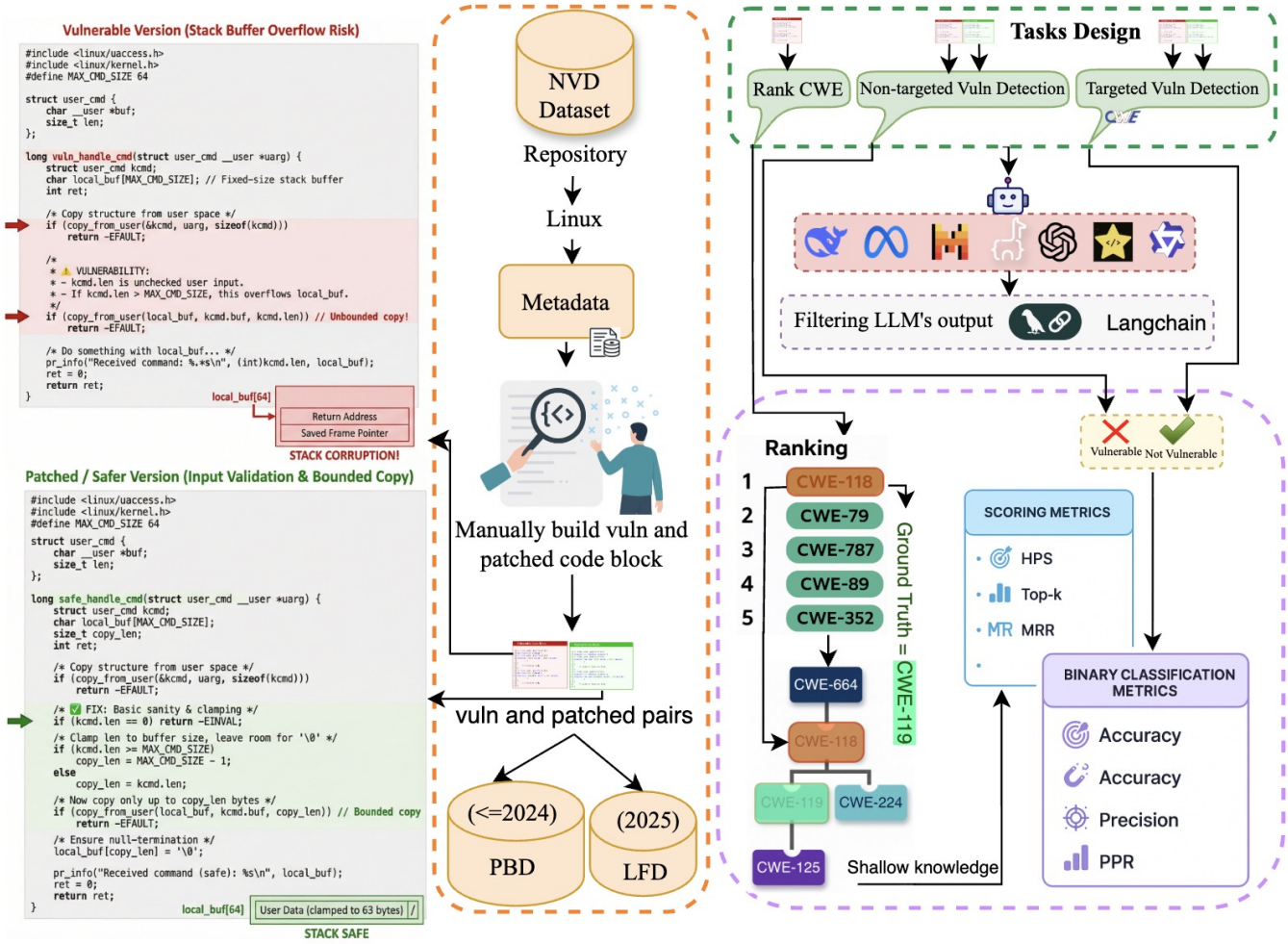


Fig. 1: LLMKernelBench Framework. Metadata (commit messages, diffs, function/file names) and vulnerable-patched code pairs are extracted from Linux CVE commits. Prompts are designed for SVD and CWE-label classification tasks in zero-shot settings. LLMs performs detection and classification. Outputs are filtered via regex. Detection is evaluated using binary and ranking metrics.

Table II summarizes the overall size and composition of the PBD and LFD after this comprehensive cleaning process. The final PBD dataset comprises a total of 314 samples and spans 74 different CWEs, while the final LFD contains 103 samples belonging to 19 different CWEs.

**CWE Distribution and Label Space.** Our dataset spans diverse CWE categories organized according to the CWE-1000 hierarchical view [33]. Table III shows the distribution of vulnerabilities across the ten fundamental CWE pillars in PBD. CWE-664 accounts for 43.3% of coverage, reflecting the prevalence of memory-related vulnerabilities in the Linux kernel. The distribution also includes 12.1% deprecated CWE identifiers (CWE-OTHER), representing historical vulnerability classifications that MITRE now marks as "PROHIBITED" for mapping. This diverse distribution ensures comprehensive evaluation across different security weakness categories, from common memory safety issues to rare configuration vulnerabilities.

TABLE II: LLMKernelBench dataset statistics

| Metric | PBD | LFD |
|---|---|---|
| Vulnerabilities | 314 | 103 |
| Unique CWEs | 74 | 19 |
| Files/Functions Changed (avg) | 1.5 / 2.0 | 1.1 / 1.3 |
| Vuln/Patched Lines (avg) | 109.4 / 114.1 | 60.9 / 63.6 |
| Lines Added/Deleted (avg) | 17.6 / 9.0 | 7.0 / 3.5 |

## B. *Tasks*

Based on this dataset, the models are evaluated across two different tasks: SVD and vulnerability type identification.

**SVD**. In this task, the models are given either a vulnerable code block (pre-fix versions from parent commits) or a non-vulnerable (patched) code block (post-fix versions from fixing commits). They are then asked to classify the code block as vulnerable (1), non-vulnerable (0), or to abstain from classifying it (-1). This third option is used because:

TABLE III: CWE pillar distribution across PBD and LFD.

| CWE-ID | Pillar Name | Coverage (%) | |
|---|---|---|---|
| | | PBD | LFD |
| CWE-664 | Improper Control of Resource Lifetime | 136 (43.3) | 62 (60.2) |
| CWE-OTHER | Other/Unmapped CWEs | 38 (12.1) | 0 (0.0) |
| CWE-682 | Incorrect Calculation | 32 (10.2) | 7 (6.8) |
| CWE-284 | Improper Access Control | 28 (8.9) | 0 (0.0) |
| CWE-691 | Insufficient Control Flow Management | 27 (8.6) | 11 (10.7) |
| CWE-707 | Improper Neutralization | 18 (5.7) | 9 (8.7) |
| CWE-693 | Protection Mechanism Failure | 16 (5.1) | 0 (0.0) |
| CWE-710 | Improper Adherence to Coding Standards | 11 (3.5) | 14 (13.6) |
| CWE-703 | Improper Check/Handling of Except. Conditions | 9 (2.9) | 0 (0.0) |
| CWE-435 | Improper Interaction Between Components | 0 (0.0) | 0 (0.0) |
| CWE-697 | Incorrect Comparison | 0 (0.0) | 0 (0.0) |

(1) real-world SVD often involves ambiguous cases where even security experts disagree; (2) it prevents LLMs from making forced binary decisions when evidence is insufficient, potentially reducing false positives and negatives; and (3) it mirrors practical security analysis workflows where analysts may need to flag code for further investigation rather than making definitive vulnerability determinations.

**CWE-label classification.** The CWE taxonomy organizes weaknesses into a hierarchical structure with four abstraction levels as shown in Figure 2: *Pillars* (highest level, 10 fundamental categories representing root security principles such as CWE-664 Improper Control of a Resource Through its Lifetime), *Classes* (general weakness types like CWE-119 Buffer Errors), *Bases* (specific weakness variants such as CWE-120 Buffer Copy without Checking Size of Input), and *Variants* (detailed platform-specific manifestations). This hierarchy is formalized in the CWE-1000 Research Concepts view [33].

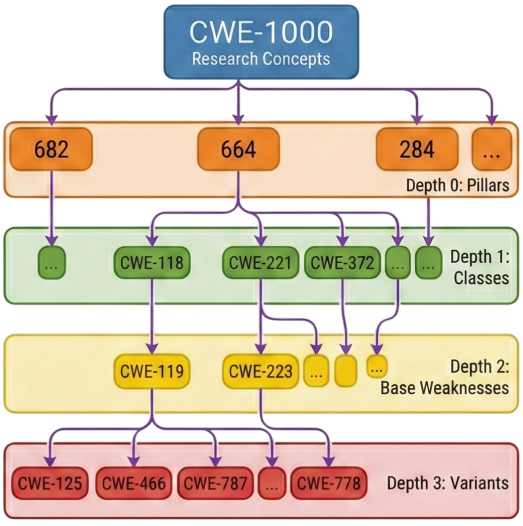


Fig. 2: CWE hierarchy.

For our CWE classification tasks, we employ a *hierarchical evaluation approach* that assesses model predictions across all abstraction levels rather than focusing solely on exact matches. This design enables us to measure the *depth of security knowledge* exhibited by LLMs, distinguishing between models that demonstrate only shallow, category-level understanding (e.g., correctly identifying a vulnerability as "resource management" but unable to specify the exact type) versus those with deep, fine-grained comprehension (e.g., pinpointing CWE-772 Missing Release of Resource after Effective Lifetime).

We introduce the HPS metric (§III-C), which quantifies how hierarchically close a predicted CWE is to the true label by computing path distance in the CWE-1000 taxonomy tree. A prediction of CWE-119 (Buffer Errors, Class level) when the true answer is CWE-121 (Stack-based Buffer Overflow, Base level) receives partial credit, as it demonstrates understanding of the vulnerability family despite missing the specific subtype. This approach addresses practical challenges in CWE classification: fine-grained labels (bases and variants) exhibit severe class imbalance, with many specific types having fewer than 10 examples in real-world datasets, rendering exact-match accuracy unreliable. Meanwhile, coarse-grained evaluation at only the pillar level (10 categories) would be too lenient, failing to distinguish between models that truly understand security concepts versus those that merely memorize broad categories.

Our hierarchical scoring thus captures a spectrum of understanding: models achieving high Top-1 accuracy (i.e., the model's first prediction is the correct label; defined in §III-C2) demonstrate precise knowledge of specific vulnerability types, while those with high HPS but low Top-1 accuracy reveal taxonomy-aware reasoning, correctly identifying the vulnerability family but struggling with fine-grained distinctions. This multi-level evaluation provides actionable insights for both model developers (to identify where models need improvement) and security practitioners (to understand whether a model's predictions are reliable enough for specific use cases).

By considering both these tasks, our benchmark systematically evaluates LLMs in terms of their ability to (1) detect the presence of vulnerabilities through binary SVD classification, and (2) identify and prioritize the most likely underlying vulnerability types through CWE-label ranking.

### C. Metrics

Due to the different nature of the two tasks, different metrics must be employed. These metrics, along with statistical significance testing, are discussed next.

*1) SVD:* To evaluate the models' performance at SVD, we consider the following terms: True Positive (TP) refers to the number of vulnerable code samples that the model correctly identifies as vulnerable; False Positive (FP) is the number of non-vulnerable samples incorrectly predicted as vulnerable; True Negative (TN) represents the number of non-vulnerable samples correctly classified as non-vulnerable; while False Negative (FN) represents the number of vulnerable samples that the model fails to identify, incorrectly classifying them as non-vulnerable. These four quantities form the basis for calculating the performance metrics described below.

It is important to note that these metrics are computed only for cases in which the models actually provided a classification, i.e., when they predicted either vulnerable (1) or

non-vulnerable (0). Samples for which the models abstained from answering (returning `-1`) are excluded from these metric calculations. In addition to reporting the standard metrics, we also provide a breakdown of how frequently each model responded versus abstained. This distinction is critical for a fair interpretation of the results (e.g., a high recall when a model only classified 5% of the samples does not carry the same significance as when it classifies 90% of the inputs). The following metrics are used:

- **Accuracy**: measures the proportion of correctly classified code samples among all samples:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \tag{1}$$

  As the dataset is relatively balanced between vulnerable and non-vulnerable samples, the accuracy accurately reflects the model's overall performance without being dominated by one class. High accuracy is important as it indicates that the model performs well across both classes.
- **Recall** – measures the proportion of actual vulnerable code samples that were correctly identified by the model:

$$\text{Recall} = \frac{TP}{TP + FN} \tag{2}$$

  In this context, it represents the number of true vulnerabilities the LLM successfully detects or flags. High recall is crucial, as missing vulnerable samples could lead to the overlooking of potential security issues.
- **Precision** – measures the proportion of code samples predicted as vulnerable that are truly vulnerable:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3}$$

  This metric indicates how many of the flagged samples are genuine vulnerabilities rather than false positives. High precision is important to reduce the number of incorrect alerts, which can waste time during manual inspection or downstream analysis.
- **Positive Prediction Rate (PPR)** – measures the proportion of all samples that the model classifies as vulnerable, regardless of correctness:

$$\text{PPR} = \frac{TP + FP}{TP + FP + TN + FN} \tag{4}$$

  While this metric is not as critical as precision or recall, it provides valuable insight into the model's potential bias toward one class. A higher PPR may indicate a tendency to over-predict vulnerabilities, whereas a lower PPR suggests under-prediction. Comparing the PPR across models helps identify whether a model systematically favors one class over the other, offering additional context for interpreting performance results.

Together, these metrics provide a comprehensive evaluation of classification performance, balancing correctness, sensitivity, and class bias.

*2) CWE-label classification:* For the task of CWE-label classification, the following metrics are employed:

- **Top-k Accuracy** [34]: Considers a prediction correct if the true vulnerability type appears among the model's k highest-ranked predictions. We measure Top-1 to Top-5 accuracy to evaluate how well models detect and rank both CVEs and CWEs. This progressive evaluation is particularly valuable in security contexts where identifying the correct vulnerability category among top candidates enables more efficient remediation efforts, even when exact classification is challenging.
- **MMR** [35]: Evaluates ranking quality by calculating:

$$\frac{1}{N} \sum_{i=1}^{N} \frac{1}{\text{rank}_i} \tag{5}$$

  Where $\text{rank}_i$ is the position of the correct answer in the ranked list for the $i$-th query. MRR rewards models that rank the correct vulnerability type higher, recognizing that, in security contexts, prioritizing findings is crucial.
- **HPS** scores a model's outputs by how close they are to the ground-truth CWE in the hierarchy. We assign weights: exact $= 1.0$, parent $= 0.8$, child $= 0.7$, sibling $= 0.6$, same-root $= 0.4$, unrelated $= 0.0$. The HPS weighting scheme follows an ordinal decay aligned with the CWE hierarchy rather than modeling precise semantic distances. Higher weights reflect closer ancestry in the taxonomy, as confusing a vulnerability with a parent or child category indicates greater understanding than predicting an unrelated weakness. The specific values are intended to preserve hierarchical ordering and distinguish near-misses from random errors, not to encode exact semantic margins. Formally, let $y_i$ be the ground truth and $(\hat{y}_{i1}, \ldots, \hat{y}_{ik})$ the top-$k$ predictions. Define $r(\hat{y}, y) \in \{\text{exact}, \text{parent}, \text{child}, \text{sibling}, \text{root}, \text{none}\}$ and a weight map $w(\cdot)$ as above. The per-sample score is

$$\text{HPS}_i = \max_{1 \le j \le k} w\big(r(\hat{y}_{ij}, y_i)\big),$$

  and the dataset score is $\text{HPS} = \frac{1}{N} \sum_{i=1}^{N} \text{HPS}_i$. HPS credits near-misses that are semantically related, which is useful for prioritization in security triage.

By combining these metrics, we obtain a holistic view of model performance, capturing not only classification accuracy and ranking quality but also the semantic proximity of predicted vulnerability categories.

*3) Statistical Significance Testing:* To determine whether observed performance differences across experimental conditions are statistically meaningful or merely due to random variation, we employ three complementary statistical approaches: statistical tests for group-level differences, degradation metrics to quantify performance changes across abstraction levels, and Pearson correlation to assess relationships between continuous variables (e.g., context window size and model degradation).

- **Statistical Test Selection**: To compare model performance across three abstraction levels (L1, L2, L3), we

follow a systematic approach to select the appropriate statistical test. First, we validate normality using the Shapiro-Wilk [36] test for each group. Second, we test homogeneity of variance using Levene's test. If both assumptions hold, we use one-way ANOVA (parametric) [37]; if only normality holds but variances differ, we use Welch's ANOVA; if normality fails, we use the Kruskal-Wallis H-test (non-parametric alternative). For PBD, all groups pass normality (Shapiro-Wilk $p>0.05$) and equal variance tests (Levene p=0.812), justifying one-way ANOVA. For LFD, Level 3 fails normality (p=0.008), requiring the Kruskal-Wallis test. Both tests evaluate whether abstraction level significantly affects accuracy, with $p<0.05$ indicating significant differences and $p\geq0.05$ suggesting no meaningful effect beyond random variation.

- **Degradation**: Measures the change in model accuracy across abstraction levels, calculated as $\text{Degradation} = \text{Accuracy}_{\text{L1}} - \text{Accuracy}_{\text{L3}}$. Positive degradation values indicate performance decline as code scope increases (higher abstraction levels), while negative values indicate improvement with broader context. This metric quantifies model robustness to code abstraction: models with near-zero degradation maintain stable performance regardless of abstraction level, while high degradation reveals fragility when processing multi-file scenarios versus isolated functions.
- **Pearson Correlation**: Measures the strength and direction of linear relationships between two variables (e.g., context window size and model degradation). The correlation coefficient (r) ranges from -1 to +1, where values close to 0 indicate no linear relationship, while values near $\pm1$ indicate strong positive or negative correlation. Statistical significance is assessed via p-value, where $p<0.05$ indicates the correlation is unlikely due to chance.
- **Mann-Whitney U Test** [38]: A non-parametric statistical test used to compare two independent groups when data do not meet the assumptions required for parametric tests (e.g., t-test). Unlike the t-test, which assumes normally distributed data with equal variances, the Mann-Whitney U test operates on ranked data and is robust to outliers and skewed distributions. The test works by ranking all observations from both groups together, then comparing the sum of ranks between groups. The U statistic represents the number of times observations from one group precede observations from the other group in the ranking. For our abstraction level analysis, we use Mann-Whitney U to compare Simple (L1) versus Complex (L2+L3) categories in LFD, where the small sample size for L3 (n=3) violates normality assumptions and creates highly unequal group sizes (85 vs 18). Statistical significance is assessed via p-value, where $p<0.05$ indicates that the observed difference between groups is unlikely due to chance alone.
- **Rank-Biserial Correlation** [39]: An effect size measure for the Mann-Whitney U test that quantifies the magnitude of difference between two groups, independent of sample size. While the p-value from Mann-Whitney U tells us whether a difference is *statistically significant*, rank-biserial correlation tells us whether that difference is *practically meaningful*. The rank-biserial correlation (r) is calculated as:

$$r = 1 - \frac{2U}{n_1 \cdot n_2} \tag{6}$$

where $U$ is the Mann-Whitney U statistic, $n_1$ is the size of the first group, and $n_2$ is the size of the second group. The correlation ranges from -1 to +1, where $r = 1$ means all observations in group 1 rank higher than all in group 2, $r = 0$ indicates no systematic difference, and $r = -1$ means all observations in group 2 rank higher. We interpret effect sizes using conventional thresholds [40]: $|r| < 0.1$ (negligible), $0.1 \leq |r| < 0.3$ (small), $0.3 \leq |r| < 0.5$ (medium), $|r| \geq 0.5$ (large). This metric is crucial for our analysis because with highly imbalanced group sizes (Simple n=85 vs Complex n=18), even small actual differences can yield statistically significant p-values purely due to sample size. Rank-biserial correlation ensures we report both statistical significance *and* practical importance.

### D. Prompt Templates

Our prompt templates for both SVD and CWE-label classification are designed with four key principles in mind: clear task specification, structured output formats, minimal introduction of bias, and scalability across vulnerability types. We focus on benchmarking the inherent LLM capabilities rather than optimizing prompting strategies, and on evaluating baseline performance to inform future research directions. Each prompt clearly states the expected task and output format, uses standardized templates for fair comparison across models, maintains essential code context while avoiding information leakage, and reflects real-world security analysis scenarios.

Representative prompt formats are shown in Figures 3 (SVD) and 4 (CWE-label classification).

**Instructions:** Check if the following C code block is vulnerable. Respond with ONLY one of these options: '1' if the code is vulnerable, '0' if the code is not vulnerable, '-1' if you are not sure if it is vulnerable or not. Do not include any explanation or additional text.

Code Block Input:

```
{code}
```

Fig. 3: Vulnerability detection and patch verification prompt.

## IV. Benchmarking Results

In this study, we leverage seven pre-trained LLMs to assess their capabilities in SVD and CWE-label classification tasks. Our selection comprises three code-specialized models (CodeLlama [41], StarCoder2 [42], and Qwen2.5-Coder [43]) explicitly trained on programming repositories,

**Instructions:** Analyze this vulnerable code and return EXACTLY 5 CWE identifiers as a JSON list. Return ONLY a JSON array in this exact format: ["['CWE-XXX", "CWE-YYY", "CWE-ZZZ", "CWE-AAA", "CWE-BBB"]. NO explanations, NO descriptions, NO additional text. Use only identifiers (CWE-followed by numbers). **Order by likelihood** (most likely first).

Code Block Input:

```
{code}
```

Fig. 4: CWE ranking prompt.

TABLE IV: LLM specs with context window sizes (tokens).

| Model | Type | # Params | Context Window |
|---|---|---|---|
| CodeLlama | Code-spec. | 7B | 16K |
| StarCoder2 | Code-spec. | 7B | 16K |
| Qwen2.5-Coder | Code-spec. | 30B | 262K |
| Llama3.1 | General | 8B | 131K |
| Mistral (v0.3) | General | 7B | 32K |
| DeepSeek-R1 | General | 7B | 131K |
| GPT-4.1-mini | General | — | 1M |

and four general-purpose models (Llama3.1 [9], Mistral [44], and DeepSeek-R1 [45], GPT-4.1-mini [46]) designed for broader language understanding tasks. This balanced selection enables us to investigate whether domain-specific training provides advantages over general language comprehension for security-critical tasks. We selected these models based on five key criteria: (1) their state-of-the-art performance on code understanding and generation benchmarks, (2) architectural diversity spanning both standard fully-dense models (e.g., CodeLlama) and Mixture of Experts (MoE) models (e.g., DeepSeek-R1) to ensure comprehensive evaluation, (3) varying parameter sizes (7B-30B and unknown for GPT-4.1-mini) and context window capacities (16K-1M tokens) to investigate how model scale and memory affect SVD capabilities, (4) balanced representation of code-specialized versus general-purpose training to assess domain expertise effects, and (5) inclusion of both smaller open-source and larger proprietary models, allowing for a fair comparison across accessibility and capability dimensions. The specifications of the models we used are detailed in Table IV.

### A. RQ1: Overall SVD performance

For this research question, we divide our analysis into two parts. First, we analyze model performance in the non-targeted setting, where models are asked to classify code as vulnerable/non-vulnerable with no information about the specific CWE. Then, we repeat the same analysis but for the targeted setting, where models are given the CWE label in the prompt. Within each of these parts, we start by looking at how often each model actually provided an answer (regardless of its correctness) and how often it refused to answer (-1), and then look into the values for the metrics (accuracy, recall, and precision).

*1) Non-targeted Classification:* **Models exhibit substantial differences in abstention behavior.** Table V shows abstention rates across both non-targeted and targeted settings. CodeLlama, GPT-4.1-mini, and Mistral consistently provide predictions, with abstention rates below 4% across all conditions. Mistral never abstains, while CodeLlama abstains in only 0.5% of non-targeted PBD cases. In contrast, Llama3.1 abstains in over half of all queries (52.6%–59.7%), refusing to commit to classifications even when CVE/CWE context is provided. DeepSeek and Qwen2.5-Coder show intermediate but asymmetric behavior: DeepSeek's abstention drops sharply from 30.4% (non-targeted) to 6.1% (targeted) in PBD, while Qwen2.5-Coder's abstention increases from 34.7% to 46.7%. Providing explicit vulnerability context does not consistently reduce abstention: Llama3.1 and Qwen2.5-Coder remain highly conservative across task types, revealing fundamental differences in model confidence calibration rather than task-specific uncertainty.

TABLE V: Abstention rates (%) for non-targeted and targeted SVD.

| | Non-Targeted | | Targeted | |
|---|---|---|---|---|
| Model | PBD (%) | LFD (%) | PBD (%) | LFD (%) |
| CodeLlama | 0.5 | 0.0 | 0.3 | 0.0 |
| DeepSeek | 30.4 | 24.3 | 6.1 | 7.3 |
| GPT-4.1-mini | 2.2 | 3.9 | 1.1 | 1.5 |
| Llama3.1 | 52.6 | 59.7 | 51.9 | 55.8 |
| Mistral | 0.0 | 0.0 | 0.0 | 0.0 |
| Qwen2.5-Coder | 34.7 | 37.9 | 46.7 | 37.9 |
| StarCoder2 | 16.9 | 22.3 | 11.3 | 21.4 |

**LLMs performed no better than random classifiers.** In the non-targeted setting, Table VI (Non-Targeted columns) presents the actual performance metrics, considering only the cases where the models did provide a classification. We can see that all models exhibit accuracies close to 50%, irrespective of architecture, size, or context window size. This performance can be seen in both the PBD and LFD, indicating that potential training data leakage had no effect on performance and that none of the models have truly learned meaningful patterns to distinguish between vulnerable and non-vulnerable code in this setting.

Consider CVE-2024-26599 (CWE-119, PBD), a clear out-of-bounds array access in a PWM driver function. The vulnerable code checks `args->args_count != 1 && args->args_count != 2` to validate input, then accesses `args->args[2]` when `args_count == 2`. This is an obvious bug: if an array has count=2, valid indices are 0 and 1 (zero-indexed), so index 2 is out of bounds. The fix simply changes `args[2]` to `args[1]`. A human reviewer can immediately identify this as vulnerable because it violates basic array indexing rules. Yet all four major models failed: Codellama and Mistral incorrectly classified both vulnerable and patched versions as vulnerable, Deepseek classified both as safe, and Llama abstained on both. This demonstrates that the dataset contains clear, detectable vulnerabilities with sufficient context, but models simply lack the fundamental security reasoning to identify even straightforward bounds violations.

```
// File: drivers/pwm/core.c
of_pwm_single_xlate(struct pwm_chip *chip,
                    const struct of_phandle_args *args) {
   struct pwm_device *pwm;

   if (chip->of_pwm_n_cells < 1)
      return ERR_PTR(-EINVAL);

   // Validates that args_count is either 1 or 2
   if (args->args_count != 1 && args->args_count != 2)
      return ERR_PTR(-EINVAL);

   pwm = pwm_request_from_chip(chip, 0, NULL);
   if (IS_ERR(pwm))
      return pwm;

   pwm->args.period = args->args[0];
   pwm->args.polarity = PWM_POLARITY_NORMAL;

   // BUG: Accessing index 2 when count==2 (valid: 0,1)
- if (args->args_count == 2 && args->args[2] &
    ↪ PWM_POLARITY_INVERTED)
+ if (args->args_count == 2 && args->args[1] &
    ↪ PWM_POLARITY_INVERTED)
      pwm->args.polarity = PWM_POLARITY_INVERSED;

   return pwm;
}
```

Fig. 5: CVE-2024-26599: out-of-bounds access; patch shown.

**Models tend to have an extreme bias towards one of the classes.** Examining metrics such as Recall, Precision, and PPR reveals consistent patterns across both the PBD and LFD. CodeLlama and Mistral stand out in both datasets with very high recall values (around 75–88% for PBD and 70–81% for LFD), indicating that they correctly flag most vulnerable samples. However, their corresponding precision and PPR values (around 50% precision and 0.8 PPR) indicate that these high recall scores are driven by a strong bias toward positive predictions: the models tend to classify most samples as vulnerable, regardless of the true label.

In contrast, models such as GPT-4.1-mini, Llama3.1, Qwen2.5-Coder, and DeepSeek exhibit the opposite behaviour across both datasets. Their low recall and PPR values indicate a strong bias toward the negative class (non-vulnerable code), with most predictions defaulting to "not vulnerable". Yet despite this conservative tendency, their precision remains low, so even when these models predict a vulnerability, they are often incorrect.

Overall, these patterns are remarkably consistent across the PBD and LFD, reinforcing the conclusion that the observed biases are intrinsic to the models rather than a result of training data leakage or dataset-specific artifacts.

*2) Targeted Classification:* Moving to the targeted classification setting, Table V presents the response distributions across models.

**Providing the CWE label influenced models' abstention behavior in different ways.** For DeepSeek and StarCoder2, including the CWE label led to fewer abstentions. Specifically, DeepSeek's proportion of predicted responses increased from 71.1% to 93.65%, while StarCoder2's increased from 81.77% to 86.21%. In contrast, for Qwen2.5-Coder, the proportion of predicted responses declined from 62.47% to 53.48%. For the remaining four models, assigning the CWE labels did not lead to substantial changes in abstention frequency. Overall, this suggests that the inclusion of CWE labels had model-dependent effects on prediction behavior. Some models became more decisive, while others grew more cautious, highlighting variability in how each model interprets and leverages the task information.

**Similar effects can be seen in the performance metrics.** In the targeted setting, Table VI (Targeted columns) summarizes the performance metrics for SVD. Overall, the accuracies remained around 50%, indicating that even when the corresponding CWE label was provided, the models did not perform better than random guessing. However, when examining the other metrics, some interesting trends emerge. For DeepSeek, recall increased from 16.0% to 44.5%, while the PPR rose from 15.7% to 48.2%. This suggests that, although overall performance did not improve, providing the label helped DeepSeek overcome its initial bias toward the negative class. A similar, albeit smaller, effect can be observed for StarCoder2, where recall increased from 18.9% to 34.3% and PPR from 21.0% to 39.1%. For Mistral, the opposite trend occurred: recall dropped from 88.2% to 20.1% and PPR from 88.9% to 23.1%. In this case, performance again did not meaningfully improve, but the bias shifted entirely from favoring the positive class to favoring the negative one. For the remaining models, no significant changes were observed across any of the metrics.

These findings once again demonstrate that the inclusion of CWE labels affects models in distinct and sometimes opposing ways, suggesting differences in how each model integrates structured contextual information during classification.

**LLMs show weak, near-chance performance across CWE pillars, reinforcing that they are not yet capable of reliable SVD classification.** In the non-targeted setting, performance varies slightly by pillar but remains close to random guessing. For PBD, accuracies range from 43.1% (CWE-707) to 55.2% (CWE-710), with most pillars clustered near 50%. For LFD, results are only available for a subset of pillars (CWE-664/682/691/707/710) and accuracies range from 40.0% to 56.1%. Beyond accuracy, the other metrics show the same overall weakness: in PBD, recall spans 28.6%–51.5% and precision stays roughly in the mid-40s to high-50s, while in LFD recall can be as low as 0.0% (CWE-691), indicating highly unstable behavior in low-support categories.

The targeted setting does not consistently improve performance. While some pillars show higher accuracy (e.g., CWE-693 in PBD rises to 58.3%, and CWE-664 in LFD rises to 55.0%), others degrade (e.g., CWE-682 in PBD drops to 44.3%, and CWE-707 in LFD drops to 51.2%). Overall, models remain near chance across pillars in both settings, as summarized in Table VII.

### *B. RQ3: Effect of Code Abstraction Level*

Real-world vulnerabilities rarely exist in isolation. A single vulnerable function often depends on context from other functions, global variables, macros, or configuration settings

TABLE VI: Performance metrics for SVD in non-targeted and targeted settings.

| Model | Non-Targeted | | | | | | | | Targeted | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | PBD (%) | | | | LFD (%) | | | | PBD (%) | | | | LFD (%) | | | |
| | Acc. | Rec. | Prec. | PPR | Acc. | Rec. | Prec. | PPR | Acc. | Rec. | Prec. | PPR | Acc. | Rec. | Prec. | PPR |
| CodeLlama | 49.0 | 75.7 | 49.4 | 76.8 | 51.0 | 70.9 | 50.7 | 69.9 | 52.2 | 92.0 | 51.2 | 89.9 | 56.8 | 92.2 | 54.0 | 85.4 |
| DeepSeek | 51.3 | 16.0 | 49.2 | 15.7 | 46.9 | 11.5 | 52.9 | 11.7 | 46.5 | 44.5 | 44.7 | 48.2 | 50.3 | 42.3 | 55.0 | 41.4 |
| GPT-4.1-mini | 50.7 | 5.8 | 60.0 | 4.9 | 51.8 | 4.2 | 66.7 | 3.1 | 49.5 | 7.8 | 49.0 | 8.0 | 53.3 | 7.3 | 77.8 | 4.6 |
| Llama3.1 | 45.0 | 3.1 | 33.3 | 5.0 | 40.9 | 0.0 | 0.0 | 0.0 | 46.7 | 0.0 | 0.0 | 0.0 | 40.9 | 0.0 | 0.0 | 0.0 |
| Mistral | 49.4 | 88.2 | 49.6 | 88.9 | 51.0 | 80.6 | 50.6 | 79.6 | 47.0 | 20.1 | 43.4 | 23.1 | 48.5 | 14.6 | 45.5 | 16.0 |
| Qwen2.5-Coder | 53.8 | 4.8 | 62.5 | 3.6 | 42.9 | 2.6 | 33.3 | 4.3 | 52.9 | 0.0 | 0.0 | 0.0 | 44.3 | 0.0 | 0.0 | 0.0 |
| StarCoder2 | 48.2 | 18.9 | 44.4 | 21.0 | 54.7 | 15.8 | 47.4 | 14.8 | 45.4 | 34.3 | 43.5 | 39.1 | 54.7 | 38.6 | 48.9 | 35.2 |

TABLE VII: Per-CWE breakdown of the performance metrics for SVD.

| CWE Pillar | Non-Targeted | | | | | | | | Targeted | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | PBD (%) | | | | LFD (%) | | | | PBD (%) | | | | LFD (%) | | | |
| | Acc. | Rec. | Prec. | PPR | Acc. | Rec. | Prec. | PPR | Acc. | Rec. | Prec. | PPR | Acc. | Rec. | Prec. | PPR |
| CWE-284 | 50.0 | 38.7 | 52.2 | 38.3 | – | – | – | – | 45.0 | 32.3 | 45.5 | 36.7 | – | – | – | – |
| CWE-664 | 47.9 | 48.0 | 50.0 | 50.0 | 48.9 | 30.2 | 45.2 | 32.1 | 51.3 | 35.1 | 50.0 | 34.2 | 55.0 | 33.3 | 55.3 | 29.0 |
| CWE-682 | 54.1 | 51.5 | 58.6 | 47.5 | 40.0 | 33.3 | 50.0 | 40.0 | 44.3 | 27.3 | 47.4 | 31.1 | 40.0 | 16.7 | 50.0 | 20.0 |
| CWE-691 | 46.2 | 28.6 | 50.0 | 30.8 | 44.4 | 0.0 | 0.0 | 0.0 | 53.8 | 35.7 | 62.5 | 30.8 | 44.4 | 20.0 | 50.0 | 22.2 |
| CWE-693 | 54.2 | 47.6 | 47.6 | 43.8 | – | – | – | – | 58.3 | 47.6 | 52.6 | 39.6 | – | – | – | – |
| CWE-703 | 49.1 | 46.4 | 48.1 | 47.4 | – | – | – | – | 47.4 | 39.3 | 45.8 | 42.1 | – | – | – | – |
| CWE-707 | 43.1 | 42.3 | 44.0 | 49.0 | 56.1 | 40.9 | 64.3 | 34.1 | 43.1 | 38.5 | 43.5 | 45.1 | 51.2 | 31.8 | 58.3 | 29.3 |
| CWE-710 | 55.2 | 38.5 | 50.0 | 34.5 | 51.7 | 40.0 | 54.5 | 37.9 | 56.9 | 42.3 | 52.4 | 36.2 | 48.3 | 33.3 | 50.0 | 34.5 |
| CWE-OTHER | 49.0 | 38.2 | 49.1 | 39.5 | – | – | – | – | 49.1 | 38.8 | 49.0 | 40.2 | – | – | – | – |

TABLE VIII: Overall performance by abstraction level: mean accuracy and standard deviation across models.

| Abstraction Level | PBD | | LFD | |
|---|---|---|---|---|
| | Mean (%) | Std. (%) | Mean (%) | Std. (%) |
| Level 1 (Single Function) | 49.6 | 0.84 | 51.3 | 1.96 |
| Level 2 (Multiple Functions) | 48.9 | 2.27 | 49.5 | 2.67 |
| Level 3 (Multiple Files) | 49.1 | 2.81 | 47.6 | 7.93 |

Statistical tests: PBD: One-way ANOVA F=0.050, p=0.9513;
LFD: Kruskal-Wallis H=0.427, p=0.8078

spread across multiple files. To capture this reality, we evaluate models at three code abstraction levels: *Level 1* presents only the modified function in isolation; *Level 2* includes all functions modified in the security patch within a single file; and *Level 3* provides the complete context across multiple files.

Our comprehensive analysis across three abstraction levels reveals surprising patterns that challenge conventional assumptions about SVD complexity. We evaluated both PBD and LFD to ensure robust findings.

**Abstraction Does Not Always Hinder Performance, But Individual Models Diverge Dramatically.** Table VIII reveals a paradox: while mean accuracy remains stable across abstraction levels (PBD: 48.9–49.6%; LFD: 47.6–51.3%), statistical tests confirm no significant difference between levels. For PBD, one-way ANOVA yields F=0.050 (p=0.9513), with the F-statistic well below 1.0, indicating that variance between abstraction levels is even smaller than within-level variance. For LFD, where normality assumptions fail (Shapiro-Wilk p=0.008 for L3), we use the non-parametric Kruskal-Wallis test (H=0.427, p=0.8078). Combined with high p-values (both $\gg$0.05), this confirms that observed accuracy differences are indistinguishable from random noise. However, the standard deviation tells a completely different story. In LFD, model agreement collapses as abstraction increases: Std grows from 1.96% at Level 1 to 7.93% at Level 3, a 4× explosion in variance. This divergence exposes that models respond in fundamentally opposite ways to increased context. Table IX quantifies this split: Codellama *improves* by 2.6% in PBD with added abstraction, yet *catastrophically degrades* by 22.0% in LFD. Meanwhile, Deepseek improves in *both* datasets ($-1.7\%$ PBD, $-6.9\%$ LFD), and Starcoder consistently struggles (5.1% PBD, 8.9% LFD degradation). The stable aggregate statistics mask wildly different model behaviors: some architectures leverage additional context effectively while others drown in it, demonstrating that abstraction robustness is model-specific rather than a universal property of LLMs.

**Model Specialization Drives Robustness.** Table IX reveals that model specialization significantly impacts abstraction sensitivity. In PBD, three models *improve* with increased abstraction: Codellama ($-2.6\%$), Deepseek ($-1.7\%$), and Qwen3 ($-0.8\%$), while Starcoder shows the highest degradation (5.1%). General-purpose models demonstrate superior robustness with 0.4% average degradation versus 8.7% for code-specialized models in PBD. This advantage becomes even more pronounced in LFD: general-purpose models maintain near-stable performance with only 0.3% degradation, while code-specialized models degrade by 8.8%, resulting in an 8.5% performance gap.

**Context Window Size Does Not Guarantee Robustness.** Table IX reveals that GPT-4.1-mini, despite having the largest context window (1M tokens), ranks only 4th with 1.0% degradation, being outperformed by models with far smaller

TABLE IX: Model robustness by abstraction sensitivity. Degradation = L1 − L3 accuracy; negative indicates improvement. Ranked by average degradation (worst → best).

| Rank (worst → best ↓) | Model | Type | Context Window | Degradation (%) PBD | LFD | Avg |
|---|---|---|---|---|---|---|
| 1 | Codellama | Code-Spec. | 16K | 22.0 | 22.0 | 22.0 |
| 2 | Starcoder | Code-Spec. | 16K | 6.1 | 4.5 | 5.3 |
| 3 | Llama | General | 128K | 4.8 | 0.0 | 2.4 |
| 4 | GPT-4.1-mini | General | 1M | 0.6 | 1.3 | 1.0 |
| 5 | Mistral | General | 32K | 0.0 | 0.0 | 0.0 |
| 6 | Qwen3 | Code-Spec. | 32K | −2.1 | 50.0* | 24.0 |
| 7 | Deepseek | General | 64K | −3.5 | 0.0 | −1.8 |
| Code-Specialized Avg. | | | | 8.7 | 8.8 | 8.8 |
| General-Purpose Avg. | | | | 0.4 | 0.3 | 0.4 |

*L1 accuracy reported (no valid L3 predictions)

TABLE X: Task-specific accuracy across abstraction levels and L1 → L3 change. Excluded abstention cases.

| Task | PBD Accuracy (%) L1 | L2 | L3 | Δ (L1→L3) | LFD Accuracy (%) L1 | L2 | L3 | Δ (L1→L3) |
|---|---|---|---|---|---|---|---|---|
| Non-Targeted | 50.7 | 49.1 | 46.9 | 3.8 | 50.8 | 48.6 | 47.2 | 3.6 |
| Targeted | 50.1 | 50.0 | 45.9 | 4.1 | 52.4 | 48.5 | 47.2 | 5.2 |

*Statistical tests (LFD, Simple vs Complex):*
Non-Targeted: Mann-Whitney U=66, p=0.0828, r=0.363 (medium effect)
Targeted: Mann-Whitney U=60, p=0.2448, r=0.257 (small effect)

contexts: Deepseek (−1.8%, 64K), Qwen3 (−2.1%, 32K), and Mistral (0.0%, 32K). Pearson correlation analysis confirms this disconnect: context window size shows no statistically significant relationship with abstraction robustness (PBD: r=−0.365, p=0.635; LFD: r=−0.592, p=0.408). Architectural design matters more than raw context capacity.

**Task-Specific Patterns Emerge.** Table X demonstrates distinct sensitivities between targeted and non-targeted tasks after correcting for -1 (no answer) handling. In PBD, both task types show similar modest degradation: Non-Targeted degrades from 50.7% (L1) to 46.9% (L3, 3.8% loss) while Targeted degrades from 50.1% to 45.9% (4.1% loss). LFD reveals comparable patterns: Non-Targeted degrades by 3.6% (50.8%→47.2%) and Targeted by 5.2% (52.4%→47.2%). Mann-Whitney U tests on the LFD Simple vs Complex comparison show that both task types exhibit performance decline with increased abstraction, with Non-Targeted showing a medium effect (U=66, p=0.0828, r=0.363) and Targeted showing a small effect (U=60, p=0.2448, r=0.257). Though neither reaches statistical significance ($p<0.05$), the consistent degradation patterns across both task types suggest that CVE/CWE context provides limited protection against abstraction complexity in this corrected analysis.

The imbalanced distribution of abstraction levels in LFD (L1: 82.5%, L2: 14.6%, L3: 2.9%) reflects natural real-world patterns but limits statistical power for fine-grained analysis. Our binary comparison (Simple vs Complex) in Table X addresses this limitation by using appropriate nonparametric methods and reporting effect sizes, but future work should include the targeted collection of multi-component, multi-file vulnerabilities to enable more robust three-way comparisons. Nevertheless, our finding that the abstraction level does *not* consistently affect average performance (as shown in Tables IX and X), combined with the dramatic model-specific divergence in robustness, provides valuable insights for deployment strategies for vulnerability detection.

Mann-Whitney U tests in Table X compare Simple (L1) vs Complex (L2 and L3) abstraction levels for the LFD dataset. Non-Targeted tasks show a medium effect size (U=66, p=0.0828, r=0.363), approaching statistical significance, while Targeted tasks show a small effect (U=60, p=0.2448, r=0.257). Both task types consistently show performance decline with increased abstraction: Targeted tasks exhibit 5.2% degradation (L1 to L3) compared to Non-Targeted tasks' 3.6% decline. Though neither reaches statistical significance ($p<0.05$), the consistent degradation across both task types indicates that CVE/CWE context provides limited robustness benefits against abstraction complexity. Notably, Targeted detection shows greater degradation despite being inherently more difficult—it requires models to identify both whether code is vulnerable *and* classify the specific vulnerability type, creating a more constrained search space. When CVE/CWE context is added, models must maintain both the specific vulnerability signature and broader code dependencies across functions and files, compounding the cognitive load. This finding suggests that semantic anchors from vulnerability-specific context alone are insufficient to compensate for the challenges of understanding vulnerabilities across multiple components, particularly in the leakage-free setting where models cannot rely on memorized CVE-code patterns.

**Code-Specialized Models Show Greater Abstraction Sensitivity.** Tables IX and X reveal significant differences between model types. In Table IX, code-specialized models show substantially worse robustness: they degrade by 8.7% in PBD and 13.3% in LFD (average 11.0%). In contrast, general-purpose models show remarkable stability: only 0.4% degradation in PBD and 0.0% in LFD (average 0.2%). This 10.8% gap demonstrates that specialization for code tasks does not inherently improve robustness to abstraction complexity. Individual models show dramatic divergence: Deepseek (general-purpose) actually *improves* by −1.8% on average as abstraction increases, while Codellama (code-specialized) degrades by 22.0% in both datasets. The consistency of Codellama's poor performance across both PBD and LFD (22.0% in each) suggests an architectural limitation rather than mere training data memorization. These findings indicate that general-purpose models' broader training may provide better transfer learning for complex code understanding tasks.

Note that, while LFD provides a valuable signal for evaluating model behavior on post-cutoff vulnerabilities, its size is inherently limited (103 samples across 19 CWEs, with very sparse coverage at higher abstraction levels). Consequently, results obtained on LFD should be interpreted as *indicative rather than definitive* of generalization capability. Observed performance differences and degradation trends highlight meaningful failure modes, but small sample sizes

may amplify variance and reduce statistical power, particularly for fine-grained analyses.

### C. RQ4: CWE Ranking Performance

Identifying the specific vulnerability type is critical for security practitioners. When a model detects vulnerable code, knowing the exact CWE category helps developers understand the root cause and apply appropriate fixes. CWE ranking evaluates whether models can not only detect vulnerabilities but also correctly classify them within the CWE taxonomy. This task is challenging because the CWE system is hierarchical, with over 900 weakness types organized into four levels: Pillars (highest-level categories like "Resource Control"), Classes, Base weaknesses, and Variants (specific implementations). We focus on root pillars, the highest level of this hierarchy, because they represent fundamental security concepts that guide remediation strategies. Even partial understanding (identifying the correct pillar when the exact variant is wrong) provides value for security triage. We evaluate models using the CWE ranking task (SVD2), where models rank their top-5 most likely CWE predictions for vulnerable code. We measure performance using three metrics: Top-k accuracy (whether the correct CWE appears in the top k predictions), MMR (how highly models rank the correct answer), and HPS (which rewards semantically similar predictions based on CWE hierarchy). Table XI summarizes performance across both PBD and LFD. **Models show Semantic Understanding of Vulnerability Families Despite Low Exact Accuracy.** Table XI reveals that precise CWE identification is far harder than binary vulnerability detection. Even the best model, GPT-4.1-mini, achieves only 2.92% Top-1 accuracy in PBD and 14.71% in LFD. Most models perform dismally: CodeLlama and DeepSeek manage less than 1% Top-1 accuracy in both datasets. However, Top-5 performance tells a more nuanced story: GPT-4.1-mini reaches 50.98% in LFD, suggesting models can narrow down possibilities even when they miss the exact CWE. Interestingly, CWE ranking exhibits mixed patterns: GPT-4.1-mini and Qwen2.5-Coder actually improve in LFD (5× and 4× better Top-1 accuracy, respectively), while code-specialized models like DeepSeek and StarCoder2 collapse to near-zero performance. The gap between HPS and Top-1 accuracy (17.32% vs. 2.92% for GPT-4.1-mini in PBD, see Table XI) indicates that models frequently predict CWEs within the correct taxonomy branch even when the exact classification is wrong. While only 13.8% of incorrect predictions are hierarchically close (distance $\leq$3, Table XV), this *taxonomy-aware prediction* has practical value for security triage, as it narrows the vulnerability family (e.g., correctly identifying buffer-related issues vs access control issues). However, the large HPS-to-Top-1 ratios (5.9× to 15.7×, Table XVII) and systematic CWE-119 bias (45.1% appearance rate, Table XVI) suggest this may reflect taxonomy pattern-matching rather than genuine semantic understanding.

**Frequent Vulnerability Types Inflate Performance Metrics.** Table XII reveals that models perform significantly better on common CWE types than rare ones. As defined in §III-C, micro accuracy weights all samples equally (favoring frequent types), while macro accuracy averages performance across all CWE classes equally. The gap between these metrics exposes frequency bias. On average, the gap is small in PBD (1.1 points) but jumps dramatically in LFD (6.1 points) (a 5.5× increase). This pattern shows data leakage hides the bias. Individual models show even stronger effects: GPT-4.1-mini achieves 51.0% micro but only 24.4% macro in LFD (26.6 gap), meaning it performs well on common vulnerabilities but fails on rare types. Qwen2.5-Coder shows similar behavior with a 9.3 gap (30.6% vs. 21.3%). Interestingly, two models show negative gaps in LFD (Llama3.1: -0.1, Mistral: -1.3), performing slightly better on rare types (possibly because they rely less on memorization). The dramatic increase in the gap from PBD to LFD confirms that models learn frequency-based patterns rather than generalizable security knowledge.

**Models Excel at Memory Safety but Fail at Configuration Vulnerabilities.** Table XIII shows models perform best on memory-safety vulnerabilities but have clear blind spots. In PBD, four out of five top performers are buffer-related (CWE-119, CWE-122, CWE-120, plus CWE-20), revealing training data bias toward common memory vulnerabilities in C/C++ codebases. CWE-119 (Buffer Errors) achieves 57.1% accuracy, CWE-122 (Heap Overflow) at 40.8%, and CWE-120 (Buffer Copy) at 23.8%. These vulnerabilities have clear syntactic patterns that models can learn. However, five CWEs remain completely undetected at 0%: CWE-327 (Broken Crypto), CWE-16 (Configuration), CWE-732 (Incorrect Permissions), CWE-1284 (Improper Validation), and CWE-326 (Inadequate Encryption). These vulnerabilities often require inter-procedural or system-level analysis that a single-function context cannot provide. LFD shows similar patterns but with lower overall performance: the best CWE-20 (Input Validation) score is only 35.7%, compared to PBD's 57.1%. Both datasets show that models struggle with semantic vulnerabilities that require domain knowledge of cryptography, permissions, and configuration, while succeeding at syntactic patterns such as buffer operations.

**CWE-119 Acts as a Semantic Magnet for Diverse Vulnerability Types.** Table XIV reveals models consistently misattribute diverse vulnerability types to CWE-119 (Buffer Errors). In PBD, all top-8 confusion pairs target CWE-119, spanning heap overflow (CWE-122, 19 instances), access control (CWE-276, 15 instances), buffer copy (CWE-120, 14 instances), and authorization issues (CWE-862, 13 instances). The diversity of source CWEs, including resource management issues such as CWE-772, CWE-399, and CWE-665, as well as integer overflow CWE-190, all incorrectly mapped to CWE-119, indicates that models exhibit a strong bias toward buffer overflow as a general-purpose prediction when encountering code complexity. LFD shows a different pattern: CWE-416 (Use After Free) dominates confusions with 57 instances, followed by CWE-119, but also spans multiple targets (CWE-123, CWE-121, CWE-20), suggesting models struggle specifically with memory lifecycle issues and map them to various memory-related categories.

**HPS Validation.** The HPS metric assumes that hierarchical

TABLE XI: CWE ranking across PBD and LFD. Columns report Top-$k$ accuracy, MRR, and HPS.

| Model | PBD: CWE Top-k (%) | | | | | LFD: CWE Top-k (%) | | | | | MRR (%) | | HPS (%) | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | $T_1$ | $T_2$ | $T_3$ | $T_4$ | $T_5$ | $T_1$ | $T_2$ | $T_3$ | $T_4$ | $T_5$ | **PBD** | **LFD** | **PBD** | **LFD** |
| CodeLlama | 0.6 | 1.3 | 1.3 | 1.3 | 1.3 | 2.0 | 2.9 | 2.9 | 2.9 | 6.9 | 1.0 | 3.4 | 5.8 | 7.1 |
| DeepSeek | 0.6 | 1.6 | 2.6 | 3.2 | 3.9 | 0.0 | 1.0 | 1.0 | 2.0 | 2.0 | 1.7 | 0.8 | 5.2 | 9.6 |
| Llama3.1 | 0.7 | 1.9 | 4.2 | 8.1 | 9.0 | 2.0 | 4.0 | 8.0 | 11.0 | 15.0 | 3.2 | 5.9 | 10.2 | 14.9 |
| Mistral | 1.0 | 1.9 | 1.9 | 2.6 | 4.5 | 5.0 | 6.0 | 8.0 | 8.0 | 9.0 | 2.8 | 6.4 | 8.3 | 9.0 |
| Qwen2.5-Coder | 1.6 | 4.9 | 8.2 | 8.8 | 11.4 | 7.1 | 15.3 | 18.4 | 26.5 | 30.6 | 5.0 | 15.1 | 14.7 | 20.9 |
| StarCoder2 | 1.3 | 3.6 | 4.8 | 6.5 | 6.5 | 0.0 | 0.0 | 0.0 | 2.0 | 4.0 | 3.3 | 0.9 | 18.0 | 25.4 |
| GPT-4.1-mini | **2.9** | **5.8** | 7.1 | 9.7 | 11.4 | 14.7 | 25.5 | 40.2 | 48.0 | 51.0 | 5.8 | 27.6 | 17.3 | 25.9 |
| **Average** | 1.3 | 2.7 | 4.2 | 5.3 | 6.7 | 2.7 | 5.3 | 5.3 | 11.6 | 11.8 | 3.2 | 5.6 | 10.6 | 14.7 |

TABLE XII: Micro/Macro accuracy analysis revealing model performance disparities across CWE types.

| **Model** | **PBD** | | | **LFD** | | |
|---|---|---|---|---|---|---|
| | **Micro (%)** | **Macro (%)** | **Gap (pp)** | **Micro (%)** | **Macro (%)** | **Gap (pp)** |
| CodeLlama | 2.2 | 2.1 | 0.1 | 7.8 | 3.2 | 4.6 |
| DeepSeek | 3.9 | 3.2 | 0.7 | 2.0 | 1.8 | 0.2 |
| Llama3.1 | 9.0 | 7.5 | 1.5 | 15.0 | 15.1 | −0.1 |
| Mistral | 7.7 | 6.1 | 1.6 | 9.0 | 10.3 | −1.3 |
| Qwen2.5-Coder | 11.4 | 8.7 | 2.7 | 30.6 | 21.3 | 9.3 |
| StarCoder2 | 6.5 | 5.1 | 1.4 | 4.0 | 0.7 | 3.3 |
| GPT-4.1-mini | 11.4 | 11.8 | −0.4 | 51.0 | 24.4 | 26.6 |
| **Average** | 7.4 | 6.4 | 1.1 | 17.1 | 11.0 | 6.1 |

TABLE XIII: Top-5 and Bottom-5 CWE Detection Performance (Averaged across all models).

| **PBD Benchmark** | | **LFD Benchmark** | |
|---|---|---|---|
| CWE ID | Acc. (%) | CWE ID | Acc. (%) |
| *Top Performers* | | *Top Performers* | |
| CWE-119 ($\uparrow_1$) | 57.1 | CWE-20 ($\uparrow_1$) | 35.7 |
| CWE-20 ($\uparrow_2$) | 44.8 | CWE-787 ($\uparrow_2$) | 31.0 |
| CWE-122 ($\uparrow_3$) | 40.8 | CWE-129 ($\uparrow_3$) | 28.6 |
| CWE-190 ($\uparrow_4$) | 31.0 | CWE-125 ($\uparrow_4$) | 25.6 |
| CWE-120 ($\uparrow_5$) | 23.8 | CWE-190 ($\uparrow_5$) | 20.0 |
| *Bottom Performers* | | *Bottom Performers* | |
| CWE-327 ($\downarrow$) | 0.0 | CWE-476 ($\downarrow$) | 12.3 |
| CWE-16 ($\downarrow$) | 0.0 | CWE-835 ($\downarrow$) | 4.8 |
| CWE-732 ($\downarrow$) | 0.0 | CWE-667 ($\downarrow$) | 2.4 |
| CWE-1284 ($\downarrow$) | 0.0 | CWE-908 ($\downarrow$) | 0.0 |
| CWE-326 ($\downarrow$) | 0.0 | CWE-401 ($\downarrow$) | 0.0 |

TABLE XIV: Top-8 misclassification patterns showing CWE-119 as a semantic magnet in PBD and CWE-416 confusion dominance in LFD.

| **PBD** | | | **LFD** | | |
|---|---|---|---|---|---|
| **True** | **Predicted** | **Count** | **True** | **Predicted** | **Count** |
| CWE-122 | CWE-119 | 19 | CWE-416 | CWE-119 | 57 |
| CWE-276 | CWE-119 | 15 | CWE-125 | CWE-119 | 31 |
| CWE-120 | CWE-119 | 14 | CWE-416 | CWE-123 | 20 |
| CWE-416 | CWE-119 | 13 | CWE-416 | CWE-121 | 19 |
| CWE-862 | CWE-119 | 13 | CWE-416 | CWE-20 | 18 |
| CWE-190 | CWE-119 | 13 | CWE-476 | CWE-119 | 18 |
| CWE-399 | CWE-119 | 13 | CWE-416 | CWE-1234 | 17 |
| CWE-772 | CWE-119 | 13 | CWE-416 | CWE-400 | 17 |

distance in the CWE-1000 taxonomy reflects semantic similarity between vulnerability types. To assess this assumption, we analyzed 2,159 CWE predictions across all models (Table XV). We found that only 13.8% (294/2,132) of incorrect predictions were hierarchically close (distance $\leq$3), suggesting most wrong predictions are taxonomically distant from the true CWE (Table XVIII). For example, when the true CWE is CWE-119 (Buffer Errors), StarCoder2 may predict CWE-121 (Stack Overflow) with hierarchy distance 2, demonstrating partial understanding of the buffer-related vulnerability family (Table XIX). However, we observed systematic bias toward predicting CWE-119: this general category appears in 45.1% of all top-5 predictions, with Llama reaching 98.1% (Table XVI). This bias inflates HPS scores without demonstrating specific vulnerability knowledge: models hedge by predicting high-abstraction parent categories that match many children. Additionally, the CWE hierarchy does not always reflect semantic similarity: CWE-129 (Array Index) and CWE-119 (Buffer Errors) are both bounds violations but have distance 7 (Table XIX), meaning semantically reasonable confusions receive low HPS credit. The large gap between HPS and Top-1 accuracy across models (Table XVII), ranging from 5.9× (GPT-4.1-mini) to 15.7× (Llama) in PBD, suggests pattern-matching on taxonomy structure rather than genuine semantic understanding. We interpret HPS as measuring *taxonomy-aware prediction* rather than deep semantic comprehension, and recommend future validation through human similarity ratings correlating hierarchy distance with expert judgments.

## V. Discussion

The conclusions in this section apply specifically to off-the-shelf, pre-trained LLMs evaluated in a zero-shot setting using standardized prompts. We do not assess the effects of task-specific fine-tuning, few-shot prompting, chain-of-thought reasoning, external tools, or hybrid analysis pipelines. Accordingly, our findings should not be interpreted as fundamental limitations of large language models as a class, but rather as evidence that current baseline LLM usage is insufficient for reliable software vulnerability detection in high-assurance settings. Note that model behavior is described using observational terminology grounded in empirical outputs. References to tendencies, biases, or memorization are intended to

characterize consistent prediction patterns rather than to imply internal model intent or underlying mechanisms.

**Key Findings.** Our benchmark exposes fundamental limitations in LLMs' security reasoning. The low CWE detection accuracy (best: 11.4% Top-5) and systematic bias toward memory-safety vulnerabilities reveal surface-level pattern recognition rather than deep security understanding. The reverse bias, where models detect patched code 13.5% better than vulnerable code, contradicts assumptions about security-conscious defaults. Performance degradation with explicit CVE/CWE context suggests additional information overwhelms rather than guides reasoning. These findings indicate that LLMs should serve as assistive tools under expert supervision, not as autonomous analyzers, and that significant architectural improvements are required for reliable vulnerability identification.

**Future Directions.** Future work should validate HPS through human expert studies. Specifically, security practitioners should rate the semantic similarity of CWE pairs on a 1-5 scale, enabling the calculation of the correlation between hierarchy distance and perceived similarity (Spearman's $\rho$). LLMKernelBench enables several research paths: (1) analyzing LLMs' reasoning processes to identify security comprehension weaknesses [47], [48]; (2) expanding to multiple languages (Python, Java, Rust, Go) to assess cross-language transfer; (3) investigating hybrid LLM-agent architectures integrating static/dynamic analysis tools; (4) examining whether LLMs introduce vulnerabilities when generating patches. These directions aim to transform our diagnostic framework into a catalyst for developing robust, security-aware models.

## VI. Threats to Validity

Although the methodology and experiments in this study were carefully designed and executed, some potential threats have to be discussed.

**Internal Validity.** One possible threat arises from the nondeterministic nature of LLMs. To minimize this effect, we set the temperature parameter to 0 in all experiments (as described in Section III), ensuring outputs were as deterministic as possible and allowing reproducible results.

Another potential source of bias lies in the use of prompting strategies, as different formulations can yield different outcomes. In this study, we focused on assessing fundamental SVD capabilities using standardized zero-shot prompts instead of optimized prompt engineering. This approach establishes a consistent and fair baseline for future research.

A further concern is that some models may have encountered PBD data during pretraining, which could affect their performance. To mitigate this, we introduced the LFD dataset, composed of vulnerabilities disclosed after the models' training cutoffs. By comparing model behavior across both datasets, we can estimate the extent to which exposure to the training data may have affected the results.

Another threat is that manual annotation was used to build the dataset, introducing an element of subjectivity. To mitigate this, clear annotation guidelines were established, and both reviewers adhered strictly to them. In cases of uncertainty, the reviewers discussed the instance, thereby reducing individual bias and ensuring consistent labeling.

Extracting only modified functions may lack the surrounding context for vulnerability detection. We mitigate this by including three abstraction levels. Our RQ3 analysis (§IV-B) validates this design by demonstrating distinct performance patterns across these context levels.

Finally, all evaluated tasks focus on static code reasoning. As a result, they do not account for dynamic execution behavior or runtime verification. While this design isolates reasoning capability from execution effects, it may limit the interpretation of results in contexts requiring dynamic analysis.

**External Validity.** Our analysis is restricted to Linux kernel C code, raising questions about its generalizability to other programming languages or domains. Nonetheless, the Linux kernel's scale, complexity, and role in security-critical systems make it a meaningful and representative benchmark for real-world challenges in understanding code.

Additionally, our evaluation does not include the most recent state-of-the-art models, such as GPT-5. However, the study includes a diverse set of seven models varying in architecture and size. The consistency of observed trends across these models, along with their alignment with the behavior of larger proprietary models such as GPT-4.1-mini, gives us confidence that our findings extend beyond the evaluated models.

**Construct Validity.** HPS relies on a manually defined weighting scheme to quantify hierarchical proximity within the CWE taxonomy. The metric is intended to preserve relative semantic ordering across hierarchy levels rather than depend on specific numeric values; accordingly, our analysis focuses on comparative trends and model rankings rather than absolute HPS magnitudes. Variations in weights that preserve hierarchy order are unlikely to affect the qualitative conclusions.

CWE assignment, particularly at fine-grained base and variant levels, involves inherent subjectivity even among experts. This is especially relevant for the LFD, where some CWEs were manually assigned following NVD-aligned guidelines, and formal inter-annotator agreement was not computed. Consequently, part of the low Top-1 accuracy in fine-grained CWE classification may reflect label ambiguity rather than purely model failure. To mitigate this, we emphasize hierarchy-aware evaluation and relative performance trends, which are more robust to label noise.

**Conclusion Validity.** Our conclusions are based on pretrained LLMs evaluated in a zero-shot setting with standardized prompts. While this enables fair and reproducible comparison, it does not capture the effects of prompt adaptation, few-shot examples, or task-specific fine-tuning. Thus, the observed limitations reflect the reliability of baseline LLM deployment rather than definitive bounds on LLM security reasoning.

Although appropriate statistical tests and effect sizes are reported, the LFD's limited size constrains statistical power, particularly at higher abstraction levels. As a result, some differences may be sensitive to sampling variance, and non-

significant results should not be interpreted as evidence of equivalence. The LFD is intended to reveal robustness trends under leakage-free conditions rather than support fine-grained statistical generalization. Finally, descriptions of model behavior (e.g., bias or memorization) refer to observed prediction patterns and do not imply internal model intent or mechanisms.

## VII. Conclusion

This paper presents LLMKernelBench, a benchmark for evaluating LLM capabilities in SVD and CWE classification using 417 real-world Linux kernel vulnerabilities. Our evaluation of seven models reveals fundamental limitations: binary vulnerability detection achieves near-random performance ($\sim$50% accuracy), CWE classification remains weak (best: 14.7% Top-1, 51.0% Top-5), and models exhibit strong bias toward memory-safety vulnerabilities. Abstraction level analysis shows model-specific responses (some improve with added context, while others degrade by up to 22%). The leakage-free dataset (LFD) reveals significant performance gaps: code-specialized models degrade by 8.8%, while general-purpose models demonstrate superior robustness with minimal degradation (0.3%). These findings demonstrate that current LLMs lack robust security reasoning and require expert supervision. LLMKernelBench provides a reproducible foundation for developing more reliable vulnerability detection tools.

## References


[1] CVE Details, "Vulnerability details and information," https://www.cvedetails.com/, 2024, accessed: 2024-06-19.

[2] K. Huang, Z. Xu, S. Yang, H. Sun, X. Li, Z. Yan, and Y. Zhang, "A survey on automated program repair techniques," *arXiv preprint arXiv:2303.18184*, 2023.

[3] Q. Zhang, C. Fang, Y. Ma, W. Sun, and Z. Chen, "A survey of learning-based automated program repair," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 2, pp. 1–69, 2023.

[4] V. J. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2312–2331, 2019.

[5] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 2123–2138.

[6] J. D. Pereira, J. R. Campos, and M. Vieira, "Machine learning to combine static analysis alerts with software metrics to detect security vulnerabilities: An empirical study," in *2021 17th European Dependable Computing Conference (EDCC)*. IEEE, 2021, pp. 1–8.

[7] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[8] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[9] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[10] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.

[11] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[12] N. Alshahwan, J. Chheda, A. Finegenova, B. Gokkaya, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang, "Automated unit test improvement using large language models at meta," *arXiv preprint arXiv:2402.09171*, 2024.

[13] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Transactions on Software Engineering*, 2024.

[14] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, "Large language models for software engineering: Survey and open problems," in *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 2023, pp. 31–53.

[15] M. L. Siddiq and J. C. Santos, "Securityeval dataset: mining vulnerability examples to evaluate machine learning-based code generation techniques," in *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*, 2022, pp. 29–33.

[16] Y. Liu, L. Gao, M. Yang, Y. Xie, P. Chen, X. Zhang, and W. Chen, "Vuldetectbench: Evaluating the deep capability of vulnerability detection with large language models," *arXiv preprint arXiv:2406.07595*, 2024.

[17] Y. Sun, D. Wu, Y. Xue, H. Liu, W. Ma, L. Zhang, M. Shi, and Y. Liu, "Llm4vuln: A unified evaluation framework for decoupling and enhancing llms' vulnerability reasoning," *arXiv preprint arXiv:2401.16185*, 2024.

[18] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the effectiveness of large language models in detecting security vulnerabilities," in *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 103–114.

[19] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *IEEE Symposium on Security and Privacy*, 2024.

[20] Z. Gao, H. Wang, Y. Zhou, W. Zhu, and C. Zhang, "How far have we gone in vulnerability detection using large language models," *arXiv preprint arXiv:2311.12420*, 2023.

[21] E. Iannone, R. Guadagni, F. Ferrucci, A. De Lucia, and F. Palomba, "The secret life of software vulnerabilities: A large-scale empirical study," *IEEE Transactions on Software Engineering*, vol. 49, no. 1, pp. 44–63, 2022.

[22] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 121–133.

[23] MITRE Corporation, "Common Weakness Enumeration (CWE)," https://cwe.mitre.org/, 2024, accessed: 2025-10-18.

[24] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2021.

[25] X. Wang, S. Wang, P. Feng, K. Sun, and S. Jajodia, "Patchdb: A large-scale security patch dataset," in *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2021, pp. 149–160.

[26] Y. Zheng, S. Pujar, B. Lewis, L. Buratti, E. Epstein, B. Yang, J. Laredo, A. Morari, and Z. Su, "D2a: A dataset built for ai-based vulnerability detection methods using differential analysis," in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2021, pp. 111–120.

[27] C. Ni, L. Shen, X. Yang, Y. Zhu, and S. Wang, "Megavul: Ac/c++ vulnerability dataset with comprehensive code representations," in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 738–742.

[28] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.

[29] G. Bhandari, A. Naseer, and L. Moonen, "Cvefixes: automated collection of vulnerabilities and their fixes from open-source software," in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*, 2021, pp. 30–39.

[30] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang *et al.*, "Codexglue: A machine learning benchmark dataset for code understanding and generation," *arXiv preprint arXiv:2102.04664*, 2021.

[31] N. I. of Standards and T. (NIST), "National vulnerability database (nvd) dashboard," https://nvd.nist.gov/general/nvd-dashboard, 2025, accessed: 2025-04-24.
[32] J. D. Pereira, J. H. Antunes, and M. Vieira, "A software vulnerability dataset of large open source c/c++ projects," in *2022 IEEE 27th Pacific Rim International Symposium on Dependable Computing (PRDC)*. IEEE, 2022, pp. 152–163.
[33] MITRE Corporation, "CWE-1000: Research Concepts," https://cwe.mitre.org/data/definitions/1000.html, 2024, accessed: 2025-10-18.
[34] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein *et al.*, "Imagenet large scale visual recognition challenge," *International journal of computer vision*, vol. 115, pp. 211–252, 2015.
[35] N. Craswell, "Mean reciprocal rank," *Encyclopedia of database systems*, pp. 1703–1703, 2009.
[36] S. S. Shapiro and M. B. Wilk, "An analysis of variance test for normality (complete samples)," *Biometrika*, vol. 52, no. 3-4, pp. 591–611, 1965.
[37] R. A. Fisher, "Statistical methods for research workers," in *Breakthroughs in statistics: Methodology and distribution*. Springer, 1970, pp. 66–70.
[38] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The annals of mathematical statistics*, pp. 50–60, 1947.
[39] D. S. Kerby, "The simple difference formula: An approach to teaching nonparametric correlation," *Comprehensive Psychology*, vol. 3, pp. 11–IT, 2014.
[40] J. Cohen, *Statistical power analysis for the behavioral sciences*, 2nd ed. Lawrence Erlbaum Associates, 1988.
[41] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
[42] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei *et al.*, "Starcoder 2 and the stack v2: The next generation," *arXiv preprint arXiv:2402.19173*, 2024.
[43] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.
[44] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," 2023. [Online]. Available: https://arxiv.org/abs/2310.06825
[45] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.
[46] OpenAI, "Gpt-4.1-mini," https://openai.com/index/gpt-4-1, 2025, large language model by OpenAI. Accessed via ChatGPT or the OpenAI API.
[47] V. Xiang, C. Snell, K. Gandhi, A. Albalak, A. Singh, C. Blagden, D. Phung, R. Rafailov, N. Lile, D. Mahan *et al.*, "Towards system 2 reasoning in llms: Learning how to think with meta chain-of-though," *arXiv preprint arXiv:2501.04682*, 2025.
[48] A. Zibaeirad and M. Vieira, "Reasoning with llms for zero-shot vulnerability detection," *arXiv preprint arXiv:2503.17885*, 2025.

## APPENDIX

This appendix provides concrete examples demonstrating the research challenges addressed in this work, followed by formal definitions of our SVD tasks and explanation of the HPS metric.

### *A. Motivating Example 1: CWE-119 Semantic Magnet Effect*

**CVE-2024-41061** (Linux kernel DRM driver, LFD post-cutoff sample)
**True CWE:** CWE-129 (Improper Validation of Array Index)
**Commit:** https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=2c93a4cd3c28
**File:** `drivers/gpu/drm/amd/display/dc/dml2 dml2_utils.c`

**Vulnerability Description:** Out-of-bounds array access in `dml2_calculate_rq_and_dlg_params()` where `out_lowest_state_idx` (value can exceed 1) is used as an index for the `FCLKChangeSupport` array with only 2 elements. The patch fixes this by always using index 0, as all elements contain identical values in the current implementation.

**GPT-4.1-mini Top-5 Predictions:** [CWE-119, CWE-125, CWE-190, CWE-416, CWE-362]

**Key Observation:** Despite this being a clear array indexing violation (CWE-129), the best-performing model systematically predicts CWE-119 (Buffer Errors) as its top choice. This exemplifies the *semantic magnet effect* where 45.1% of all model predictions collapse into CWE-119 (Table XVI), inflating HPS scores without demonstrating fine-grained vulnerability understanding. The hierarchy distance between CWE-129 and CWE-119 is 7—semantically similar (both are bounds violations) but taxonomically distant, resulting in low HPS credit despite reasonable confusion.

### *B. Motivating Example 2: Patched Code Misclassified as Vulnerable*

**CVE-2024-40994** (Linux kernel PTP driver)
**True CWE:** CWE-190 (Integer Overflow)
**Commit:** https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=1e197e5d7d4f
**File:** `drivers/ptp/ptp_sysfs.c`

**Vulnerability Description:** Integer overflow in allocation size calculation. The vulnerable code computes `size = sizeof(int) * max` before calling `kzalloc(size, GFP_KERNEL)`, allowing overflow when `max` is large. The patch replaces this with `kcalloc(max, sizeof(int), GFP_KERNEL)`, which includes built-in overflow checking.

**GPT-4.1-mini Binary Classification:**

- **Vulnerable code:** IS_VULNERABLE = 0 (False Negative)
- **Patched code:** IS_VULNERABLE = 1 (False Positive)

**Key Observation:** The model fails to detect the subtle integer overflow in the original code but incorrectly flags the *fixed* version as vulnerable. This reverse bias occurs because `kcalloc` is a repair idiom often associated with vulnerability fixes. Models learn surface-level syntactic patterns ("allocation function replacement = security issue") rather than understanding the semantic correctness of the code change. This failure mode directly motivates our multi-task evaluation framework spanning vulnerability detection and CWE classification.

### *C. CWE Hierarchy and HPS Metric*

The CWE hierarchy organizes vulnerabilities across four abstraction levels:

- **Pillar**: Highest-level category (e.g., CWE-664: Improper Control of Resource Lifetime)
- **Class**: Broader weakness category (e.g., CWE-119: Improper Restriction of Operations within Memory Buffer Bounds)

TABLE XV: Hierarchy distance analysis for incorrect CWE predictions across models. "Close≤3" indicates predictions within hierarchy distance 3 of the true CWE. Statistics computed over N predictions per model.

| Model | N | Wrong | Close≤3 | Close (%) | Avg Dist |
|---|---|---|---|---|---|
| Llama | 310 | 308 | 53 | 17.2 | 4.65 |
| GPT-4.1-mini | 308 | 299 | 55 | 18.4 | 4.67 |
| Starcoder | 310 | 306 | 59 | 19.3 | 4.60 |
| Mistral | 310 | 307 | 32 | 10.4 | 4.87 |
| Deepseek | 311 | 309 | 38 | 12.3 | 5.22 |
| Qwen2.5-Coder | 300 | 295 | 27 | 9.2 | 6.03 |
| Codellama | 310 | 308 | 30 | 9.7 | 5.96 |
| **Overall** | 2159 | 2132 | 294 | 13.8% | 5.14 |

TABLE XVI: Frequency of CWE-119 appearing in top-5 predictions across models, indicating systematic bias toward this general Buffer Errors category.

| Model | CWE-119 in Top-5 | Total Pred. | Rate (%) |
|---|---|---|---|
| Llama | 304 | 310 | 98.1 |
| Starcoder | 259 | 310 | 83.5 |
| GPT-4.1-mini | 223 | 308 | 72.4 |
| Qwen2.5-Coder | 102 | 300 | 34.0 |
| Deepseek | 60 | 311 | 19.3 |
| Mistral | 21 | 310 | 6.8 |
| Codellama | 5 | 310 | 1.6 |
| **Overall** | 974 | 2159 | 45.1 |

- **Base**: Specific vulnerability type (e.g., CWE-787: Out-of-bounds Write)
- **Variant**: Implementation-specific manifestation (e.g., CWE-120: Buffer Copy without Checking Size of Input)

**HPS Calculation Example:** Consider CVE-2022-26490 (CWE-120). An LLM predicting different hierarchy levels receives graduated credit:

- **Exact match** (CWE-120) → HPS = 1.0
- **Parent** (CWE-787) → HPS = 0.8 (distance 1)
- **Grandparent** (CWE-119) → HPS = 0.7 (distance 2)
- **Unrelated** (CWE-284: Access Control) → HPS = 0.0 (distance >5)

This graduated scoring reveals security reasoning depth: predicting CWE-119 instead of CWE-120 still demonstrates understanding of memory safety issues, even if lacking precision about the specific buffer operation. Our validation analysis (Table XV) shows that only 13.8% of incorrect predictions are hierarchically close (distance ≤3), suggesting models primarily pattern-match taxonomy structure rather than exhibiting genuine semantic understanding.

TABLE XVII: Gap between HPS and Top-1 accuracy showing potential taxonomy pattern-matching. Ratio indicates how many times larger HPS is compared to Top-1. Data from Table XI.

| Model | PBD | | | LFD | | |
|---|---|---|---|---|---|---|
| | Top-1 (%) | HPS (%) | Ratio (×) | Top-1 (%) | HPS (%) | Ratio (×) |
| Llama | 0.7 | 10.2 | 15.7 | 2.0 | 14.9 | 7.5 |
| Starcoder | 1.3 | 18.0 | 13.9 | 0.0 | 25.4 | ∞ |
| Qwen2.5-Coder | 1.6 | 14.7 | 9.0 | 7.1 | 20.9 | 2.9 |
| Codellama | 0.6 | 5.8 | 9.1 | 2.0 | 7.1 | 3.6 |
| Mistral | 1.0 | 8.3 | 8.5 | 5.0 | 9.0 | 1.8 |
| Deepseek | 0.6 | 5.2 | 8.1 | 0.0 | 9.6 | ∞ |
| GPT-4.1-mini | 2.9 | 17.3 | 5.9 | 14.7 | 25.9 | 1.8 |

TABLE XVIII: Distribution of Hierarchy Distances for Incorrect CWE Predictions with measurable distances ($N = 1836$ of 2132 total incorrect predictions). 16.0% of these errors fall within a proximity of $\leq 3$ (or 13.8% of all 2132 incorrect predictions).

| Distance | Count | Prop. (%) | Cumul. (%) |
|---|---|---|---|
| 1 | 52 | 2.8 | 2.8 |
| 2 | 131 | 7.1 | 10.0 |
| 3 | 111 | 6.0 | 16.0 |
| 4 | 305 | 16.6 | 32.6 |
| 5 | 427 | 23.3 | 55.9 |
| 6 | 401 | 21.8 | 77.7 |
| 7 | 275 | 15.0 | 92.7 |
| 8 | 99 | 5.4 | 98.1 |
| 9 | 35 | 1.9 | 100.0 |
| **Total $\leq 3$** | **294** | **16.0** | – |

TABLE XIX: Illustrative CWE Prediction Patterns and HPS Credit Assignments based on CWE-1000 Taxonomy Distance.

| Ground Truth | Prediction | Dist. | Relationship |
|---|---|---|---|
| CWE-119 | CWE-121 | 2 | Child → Parent |
| CWE-120 | CWE-119 | 2 | Child → Parent |
| CWE-125 | CWE-119 | 3 | Cousin |
| CWE-416 | CWE-119 | 4 | Distant Type |
| CWE-129 | CWE-119 | 7 | Distant Branch |

*Note:* Semantically similar weaknesses (e.g., CWE-129 and CWE-119) may receive lower HPS credit if located on disparate taxonomy branches.